

2367. Number of Arithmetic Triplets

Easy

Topics

Companies

Hint

You are given a **0-indexed, strictly increasing** integer array `nums` and a positive integer `diff`. A triplet `(i, j, k)` is an **arithmetic triplet** if the following conditions are met:

- `i < j < k`,
- `nums[j] - nums[i] == diff`, and
- `nums[k] - nums[j] == diff`.

Return the number of unique **arithmetic triplets**.

Example 1:

Input: `nums = [0,1,4,6,7,10]`, `diff = 3`

Output: 2

Explanation:

`(1, 2, 4)` is an arithmetic triplet because both `7 - 4 == 3` and `4 - 1 == 3`.

`(2, 4, 5)` is an arithmetic triplet because both `10 - 7 == 3` and `7 - 4 == 3`.

Example 2:

Input: `nums = [4,5,6,7,8,9]`, `diff = 2`

Output: 2

Explanation:

`(0, 2, 4)` is an arithmetic triplet because both `8 - 6 == 2` and `6 - 4 == 2`.

`(1, 3, 5)` is an arithmetic triplet because both `9 - 7 == 2` and `7 - 5 == 2`.

Code

Java Auto

```
2 public int arithmeticTriplets(int[] nums, int diff) {
3     Set<Integer> set = new HashSet<>();
4
5     for (int num : nums) {
6         set.add(num);
7     }
8
9     int count = 0;
10
11    for (int num : nums) {
12        if (set.contains(num + diff) && set.contains(num + 2 * diff)) {
13            count++;
14        }
15    }
16
17    return count;
18 }
```

Saved

Ln 1, Col 1

Testcase Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =

`[0,1,4,6,7,10]`

diff =

3

728. Self Dividing Numbers

Easy

Topics

Companies

Hint

A **self-dividing number** is a number that is divisible by every digit it contains.

- For example, `128` is a **self-dividing number** because `128 % 1 == 0`, `128 % 2 == 0`, and `128 % 8 == 0`.

A **self-dividing number** is not allowed to contain the digit zero.

Given two integers `left` and `right`, return a list of all the **self-dividing numbers** in the range `[left, right]` (both **inclusive**).

Example 1:

Input: `left = 1, right = 22`

Output: `[1,2,3,4,5,6,7,8,9,11,12,15,22]`

Example 2:

Input: `left = 47, right = 85`

Output: `[48,55,66,77]`

Constraints:

- `1 <= left <= right <= 104`

Discover more

Online Learning Platform

2K

31

☆

↗

?

17 Online

</> Code

Java ▾ 🔒 Auto

≡ 📖 {} ↺ ↻

```

1  class Solution {
2      public List<Integer> selfDividingNumbers(int left, int right) {
3          List<Integer> result = new ArrayList<>();
4
5          for (int i = left; i <= right; i++) {
6              if (isSelfDividing(i)) {
7                  result.add(i);
8              }
9          }
10
11         return result;
12     }
13
14     private boolean isSelfDividing(int num) {
15         int temp = num;
16
17         while (temp > 0) {

```

Saved

Ln 29, Col 2

☑ Testcase | >_ Test Result

Accepted

Runtime: 0 ms

☑ Case 1

☑ Case 2

Input

left =

1

right =

22

2427. Number of Common Factors

Easy

Topics

Companies

Hint

Given two positive integers a and b , return the number of **common** factors of a and b .

An integer x is a **common factor** of a and b if x divides both a and b .

Example 1:

Input: $a = 12, b = 6$

Output: 4

Explanation: The common factors of 12 and 6 are 1, 2, 3, 6.

Example 2:

Input: $a = 25, b = 30$

Output: 2

Explanation: The common factors of 25 and 30 are 1, 5.

Constraints:

- $1 \leq a, b \leq 1000$



Great ideas deserve
a **great domain** name.

Seen this question in a real interview before? 1/6

Code

Java Auto

```
1 class Solution {
2     public int commonFactors(int a, int b) {
3         int count = 0;
4         int limit = Math.min(a, b);
5
6         for (int i = 1; i <= limit; i++) {
7             if (a % i == 0 && b % i == 0) {
8                 count++;
9             }
10        }
11
12        return count;
13    }
14 }
```

Saved

Ln 12, Col 22

☒ Testcase | >_ Test Result**Accepted** Runtime: 0 ms☒ Case 1☒ Case 2

Input

a =

12

b =

6

1342. Number of Steps to Reduce a Number to Zero

Easy | Topics | Companies | Hint

Given an integer `num`, return the number of steps to reduce it to zero.

In one step, if the current number is even, you have to divide it by 2, otherwise, you have to subtract 1 from it.

Example 1:

Input: num = 14
Output: 6
Explanation:
Step 1) 14 is even; divide by 2 and obtain 7.
Step 2) 7 is odd; subtract 1 and obtain 6.
Step 3) 6 is even; divide by 2 and obtain 3.
Step 4) 3 is odd; subtract 1 and obtain 2.
Step 5) 2 is even; divide by 2 and obtain 1.
Step 6) 1 is odd; subtract 1 and obtain 0.

Example 2:

Input: num = 8
Output: 4
Explanation:
Step 1) 8 is even; divide by 2 and obtain 4.
Step 2) 4 is even; divide by 2 and obtain 2.
Step 3) 2 is even; divide by 2 and obtain 1.
Step 4) 1 is odd; subtract 1 and obtain 0.

Example 3:

Code

Java Auto

```
1 class Solution {
2     public int numberOfSteps(int num) {
3         int steps = 0;
4
5         while (num > 0) {
6             if (num % 2 == 0) {
7                 num /= 2;
8             } else {
9                 num -= 1;
10            }
11            steps++;
12        }
13
14        return steps;
15    }
16 }
```

Saved Upgrade to Cloud Saving Ln 14, Col 22

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

num =
14

Output

6

263. Ugly Number

Easy | Topics | Companies

An **ugly number** is a *positive* integer which does not have a prime factor other than 2, 3, and 5.

Given an integer `n`, return `true` if `n` is an **ugly number**.

Example 1:

Input: `n = 6`
Output: `true`
Explanation: `6 = 2 × 3`

Example 2:

Input: `n = 1`
Output: `true`
Explanation: 1 has no prime factors.

Example 3:

Input: `n = 14`
Output: `false`
Explanation: 14 is not ugly since it includes the prime factor 7.

Constraints:

- $-2^{31} \leq n \leq 2^{31} - 1$

Code

Java | Auto

```
4         return false;
5     }
6
7     while (n % 2 == 0) {
8         n /= 2;
9     }
10
11    while (n % 3 == 0) {
12        n /= 3;
13    }
14
15    while (n % 5 == 0) {
16        n /= 5;
17    }
18
19    return n == 1;
20 }
```

Saved

Ln 19, Col 23

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

n =

6

Output

true

1688. Count of Matches in Tournament

Easy

Topics

Companies

Hint

You are given an integer n , the number of teams in a tournament that has strange rules:

- If the current number of teams is **even**, each team gets paired with another team. A total of $n / 2$ matches are played, and $n / 2$ teams advance to the next round.
- If the current number of teams is **odd**, one team randomly advances in the tournament, and the rest gets paired. A total of $(n - 1) / 2$ matches are played, and $(n - 1) / 2 + 1$ teams advance to the next round.

Return the number of matches played in the tournament until a winner is decided.

Example 1:

Input: $n = 7$

Output: 6

Explanation: Details of the tournament:

- 1st Round: Teams = 7, Matches = 3, and 4 teams advance.
 - 2nd Round: Teams = 4, Matches = 2, and 2 teams advance.
 - 3rd Round: Teams = 2, Matches = 1, and 1 team is declared the winner.
- Total number of matches = $3 + 2 + 1 = 6$.

Example 2:

Input: $n = 14$

Output: 13

Explanation: Details of the tournament:

- 1st Round: Teams = 14, Matches = 7, and 7 teams advance.
- 2nd Round: Teams = 7, Matches = 3, and 4 teams advance.
- 3rd Round: Teams = 4, Matches = 2, and 2 teams advance.

</> Code

Java Auto

```

1  class Solution {
2      public int numberOfMatches(int n) {
3          int matches = 0;
4
5          while (n > 1) {
6              if (n % 2 == 0) {
7                  matches += n / 2;
8                  n /= 2;
9              } else {
10                 matches += (n - 1) / 2;
11                 n = (n - 1) / 2 + 1;
12             }
13         }
14
15         return matches;
16     }
17 }
    
```

Saved

Ln 15, Col 24

☒ Testcase |
 >_ Test Result

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

$n =$

7

Output

6

1450. Number of Students Doing Homework at a Given Time

Easy

Topics

Companies

Hint

Given two integer arrays `startTime` and `endTime` and given an integer `queryTime`.

The `i`th student started doing their homework at the time `startTime[i]` and finished it at time `endTime[i]`.

Return *the number of students* doing their homework at time `queryTime`. More formally, return the number of students where `queryTime` lays in the interval `[startTime[i], endTime[i]]` inclusive.

Example 1:

Input: `startTime = [1,2,3]`, `endTime = [3,2,7]`, `queryTime = 4`

Output: 1

Explanation: We have 3 students where:

The first student started doing homework at time 1 and finished at time 3 and wasn't doing anything at time 4.

The second student started doing homework at time 2 and finished at time 2 and also wasn't doing anything at time 4.

The third student started doing homework at time 3 and finished at time 7 and was the only student doing homework at time 4.

Example 2:

Input: `startTime = [4]`, `endTime = [4]`, `queryTime = 4`

Output: 1

Explanation: The only student was doing their homework at the `queryTime`.

Code

Java Auto

```
1 class Solution {
2     public int busyStudent(int[] startTime, int[] endTime, int queryTime) {
3         int count = 0;
4
5         for (int i = 0; i < startTime.length; i++) {
6             if (startTime[i] <= queryTime && queryTime <= endTime[i]) {
7                 count++;
8             }
9         }
10
11         return count;
12     }
13 }
```

Saved

Ln 11, Col 22

☒ Testcase | Test Result**Accepted** Runtime: 0 ms☒ Case 1☒ Case 2

Input

startTime =

`[1,2,3]`

endTime =

`[3,2,7]`

657. Robot Return to Origin

Solved

Easy

Topics

Companies

There is a robot starting at the position `(0, 0)`, the origin, on a 2D plane. Given a sequence of its moves, judge if this robot **ends up at** `(0, 0)` after it completes its moves.

You are given a string `moves` that represents the move sequence of the robot where `moves[i]` represents its `ith` move. Valid moves are `'R'` (right), `'L'` (left), `'U'` (up), and `'D'` (down).

Return `true` if the robot returns to the origin after it finishes all of its moves, or `false` otherwise.

Note: The way that the robot is "facing" is irrelevant. `'R'` will always make the robot move to the right once, `'L'` will always make it move left, etc. Also, assume that the magnitude of the robot's movement is the same for each move.

Example 1:

Input: `moves = "UD"`

Output: `true`

Explanation: The robot moves up once, and then down once. All moves have the same magnitude, so it ended up at the origin where it started. Therefore, we return true.

Example 2:

Input: `moves = "LL"`

Output: `false`

Explanation: The robot moves left twice. It ends up two "moves" to the left of the origin. We return false because it is not at the origin at the end of its moves.

Code

Java Auto

```
1 class Solution {
2     public boolean judgeCircle(String moves) {
3         int x = 0;
4         int y = 0;
5
6         for (char move : moves.toCharArray()) {
7             if (move == 'R') {
8                 x++;
9             } else if (move == 'L') {
10                x--;
11            } else if (move == 'U') {
12                y++;
13            } else if (move == 'D') {
14                y--;
15            }
16        }
17        return x == 0 && y == 0;
```

Saved

Ln 5, Col 1

Testcase | Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

`moves =`

`"UD"`

Output

`true`

1523. Count Odd Numbers in an Interval Range

[Easy](#) [Topics](#) [Companies](#) [Hint](#)

Given two non-negative integers `low` and `high`. Return the *count of odd numbers between* `low` and `high` (*inclusive*).

Example 1:

Input: `low = 3, high = 7`
Output: 3
Explanation: The odd numbers between 3 and 7 are [3,5,7].

Example 2:

Input: `low = 8, high = 10`
Output: 1
Explanation: The odd numbers between 8 and 10 are [9].

Constraints:

- $0 \leq \text{low} \leq \text{high} \leq 10^9$

[Discover more](#) [Programming Language Tutorials](#)

Seen this question in a real interview before? 1/6

</> Code

Java Auto

```
1 class Solution {
2     public int countOdds(int low, int high) {
3         int count = (high - low) / 2;
4
5         if (low % 2 == 1 || high % 2 == 1) {
6             count++;
7         }
8
9         return count;
10    }
11 }
```

Saved

Ln 9, Col 22

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

low =
3

high =
7

2129. Capitalize the Title

Easy

Topics

Companies

Hint

You are given a string `title` consisting of one or more words separated by a single space, where each word consists of English letters. **Capitalize** the string by changing the capitalization of each word such that:

- If the length of the word is 1 or 2 letters, change all letters to lowercase.
- Otherwise, change the first letter to uppercase and the remaining letters to lowercase.

Return *the capitalized* `title`.

Example 1:

Input: `title = "capiTalIze tHe title"`

Output: `"Capitalize The Title"`

Explanation:

Since all the words have a length of at least 3, the first letter of each word is uppercase, and the remaining letters are lowercase.

Example 2:

Input: `title = "First leTTeR of EACH Word"`

Output: `"First Letter of Each Word"`

Explanation:

The word "of" has length 2, so it is all lowercase.

The remaining words have a length of at least 3, so the first letter of each remaining word is uppercase, and the remaining letters are lowercase.

Example 3:

Input: `title = "i lOve leetcode"`

Code

Java Auto

```

6       for (int i = 0; i < words.length; i++) {
7           String word = words[i].toLowerCase();
8
9           if (word.length() > 2) {
10              word = Character.toUpperCase(word.charAt(0)) + word.substring(1);
11          }
12
13          result.append(word);
14
15          if (i != words.length - 1) {
16              result.append(" ");
17          }
18      }
19
20      return result.toString();
21  }
22 }
```

Saved

Ln 20, Col 34

Testcase
Test Result

Accepted

Runtime: 2 ms

Case 1

Case 2

Case 3

Input

title =
"capITalIze tHe titLe"

Output

"Capitalize The Title"